

Der Bentley–Ottmann-Algorithmus

Handout zur Seminarpräsentation

Ross Alexander Khrisna
Universität Kassel – Fachbereich 16

1. Problemdefinition

Gegeben sei eine Menge von Liniensegmenten

$$S = \{s_1, \dots, s_n\} \subset \mathbb{R}^2$$

mit

$$s_i = \{P_i + \lambda(Q_i - P_i) \mid 0 \leq \lambda \leq 1\}.$$

Je nach Fragestellung unterscheidet man:

- **Entscheidungsproblem:** Existiert ein Schnittpunkt?
- **Reporting-Problem:** Bestimme die Menge der Schnittpunkte

$$I = \{p \in \mathbb{R}^2 \mid \exists s_i \neq s_j : p \in s_i \cap s_j\}$$

- **Counting-Problem:** Bestimme $k = |I|$.

Der Bentley–Ottmann-Algorithmus löst das **Reporting-Problem** und damit auch das **Counting-Problem**.

2. Naiver Ansatz

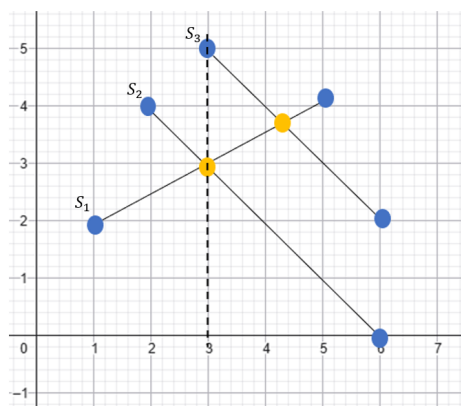
Der naive Algorithmus prüft jedes ungeordnete Segmentpaar:

$$\binom{n}{2} = \frac{n(n-1)}{2} \implies T(n) = \Theta(n^2).$$

Diese quadratische Laufzeit ist für große n unpraktikabel.

3. Sweep-Line-Technik

Grundidee: Eine vertikale Linie bewegt sich von links nach rechts über die Ebene. Es werden nur Segmente betrachtet, die die Sweep-Line aktuell schneiden.



Wesentliche Beobachtung: Ein neuer Schnittpunkt kann nur zwischen benachbarten Segmenten im Sweep-Line-Status entstehen.

Verarbeitete Ereignisse beim Shamos-Hoey:

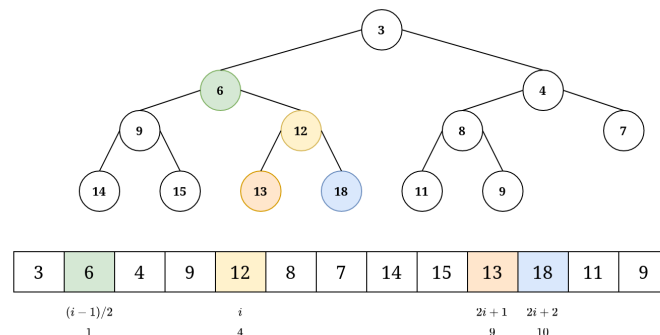
1. Linker Endpunkt (Einfügen in Status)
2. Rechter Endpunkt (Löschen aus Status)

4. Shamos–Hoey-Algorithmus

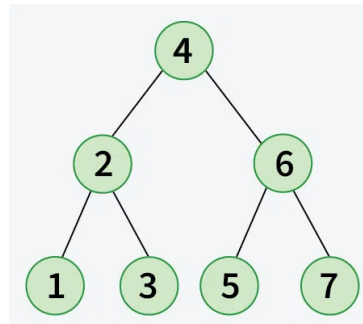
Löst das Entscheidungsproblem in $O(n \log n)$.

Verwendete Datenstrukturen:

- **Event-Queue:** Prioritätswarteschlange (Min-Heap) nach x -Koordinate.



- **Sweep-Line-Status:** Balancierter binärer Suchbaum (Baumhöhe $h \in O(\log n)$).



5. Bentley–Ottmann-Algorithmus

Erweiterung um ein drittes Ereignis:

- **Schnittpunkt** zweier aktiver Segmente.

Wird ein neuer Schnittpunkt erkannt:

- Berechnung der Koordinaten & Einfügen in die Event-Queue.
- Aktualisierung der Sweep-Line-Ordnung (SWAP).

Operationen im Sweep-Line-Status: INSERT, DELETE, PRED, SUCC, SWAP mit Kosten von jeweils $O(\log n)$.

6. Geometrische Prädikate

Orientierungstest (ccw)

$$\text{ccw}(P_1, P_2, P_3) = \text{sgn} \left(\det \begin{pmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{pmatrix} \right).$$

Zwei Segmente AB und CD schneiden sich im Inneren genau dann, wenn:

$$\text{ccw}(A, B, C) \cdot \text{ccw}(A, B, D) < 0 \quad \text{und} \quad \text{ccw}(C, D, A) \cdot \text{ccw}(C, D, B) < 0.$$

7. Laufzeitanalyse

Die Ereignismenge besteht aus $|E| = 2n + k$. Da pro Ereignis $O(\log n)$ Kosten anfallen, ergibt sich:

$$T(n, k) = O((n + k) \log n).$$

Diese Schranke ist optimal bezüglich des Sortieraufwands $\Omega(n \log n)$ und der Ausgabe $\Omega(k)$.

8. Spezialisierungen

- **Orthogonale Segmente:** Unterteilung in H (horizontal) und V (vertikal). Es verbleiben lediglich $|H| \cdot |V|$ potenzielle Schnittpunktpaare.
- **Numerische Stabilität:** Gleitkommafehler können die Ordnung zerstören. Lösungen: Exakte Arithmetik oder Snap-Rounding (z. B. in CGAL).

9. Ausblick

- **Chazelle & Edelsbrunner (1992):** $O(n \log n + k)$.
- **Balaban (1995):** Gleiche Laufzeit, geringerer Speicherbedarf.

Literatur

Bentley, J. L.; Ottmann, T. A. (1979). Algorithms for Reporting and Counting Geometric Intersections.

Shamos, M.; Hoey, D. (1976). Geometric Intersection Problems.

Chazelle, B.; Edelsbrunner, H. (1992). An Optimal Algorithm for Intersecting Line Segments.

Balaban, I. J. (1995). An Optimal Algorithm for Finding Segment Intersections.

Bildquellen

<https://www.r-jin.dev/writings/heaps>

<https://media.geeksforgeeks.org/wp-content/uploads/20241004102053776744/Balaban-a-Binary-Search-Tree-4.webp>